

Informe técnico ADESSO-365

Generación automática · 2026-05-15

1. Identificación del producto

ADESSO-365 es una aplicación web orientada a la administración de conjuntos residenciales (condominios), implementada en el repositorio condomanager-pro. Centraliza información de complejos, unidades, residentes, incidentes, reservas de amenidades, facturación interna, documentos, comunicaciones y cobros de suscripción de la plataforma mediante integración con Recurrente.

2. Resumen ejecutivo

La solución sigue el modelo de una SPA sobre Next.js con App Router, autenticación de sesión (NextAuth), persistencia relacional con Prisma sobre MySQL alojado en Railway (período gratuito limitado a 30 días según el proveedor), internacionalización (next-intl, locales es/en) y despliegue típico como proceso Node.js (next start). Los administradores de plataforma configuran claves Recurrente y datos de cobro en base de datos; el middleware aplica reglas de suscripción para rutas API sensibles.

3. Stack tecnológico

- Framework: Next.js 16.x (React 19).
- Lenguaje: TypeScript.
- Estilos: Tailwind CSS v4.
- ORM: Prisma 5.x sobre MySQL alojado en Railway (solo 30 días gratis según condiciones del proveedor).
- Auth: NextAuth v5 beta.
- i18n: next-intl.
- Correo: Nodemailer con SMTP de Brevo (plan gratuito).
- Pagos plataforma: API Recurrente (checkout, webhooks).
- Almacenamiento de objetos: AWS S3 vía SDK.
- Validación: Zod; formularios: react-hook-form.
- PDF cliente (reportes UI): jsPDF + jspdf-autotable.
- Otros: Chart.js, web-push, ExcelJS, Framer Motion.

4. Arquitectura de aplicación

Las rutas públicas y privadas viven bajo `src/app/[locale]/...`. Las API Route Handlers están en `src/app/api/...`. La capa `lib/` agrupa políticas de negocio compartidas (facturación plataforma, Recurrente, correo público de soporte, reglas de suscripción). Los componentes de interfaz se agrupan por dominio en `src/components/`. El archivo `src/middleware.ts` encadena NextAuth con el middleware de next-intl y aplica controles adicionales para `/api` cuando corresponde.

5. Middleware y archivos estáticos

El matcher del middleware excluye `_next/static`, `_next/image`, `favicon.ico`, `manifest.json`, `sw.js`, `uploads` y cualquier ruta cuyo último segmento contiene un punto (patrón `.*\.`), de modo que activos como `logoadesso.svg` no pasan por la cadena `/auth`. Para rutas `/dashboard` sin sesión se redirige a `login` preservando el prefijo de idioma cuando existe.

5.1 Control de suscripción en API

Para rutas `/api/` que no están en la lista de exenciones (`src/lib/platform-subscription-rules.ts`), el middleware reenvía internamente la cookie de sesión a `GET /api/platform-fee/access-for-session`. Si esa verificación devuelve 403, la respuesta se propaga al cliente, bloqueando operaciones cuando la suscripción de plataforma del complejo no está vigente según las reglas de negocio implementadas.

6. Autenticación, roles y soporte

Los usuarios tienen rol (enum `Role` en Prisma), estatus y vínculo opcional a un complejo como personal o administrador. El acceso a la ruta `/support` está restringido en código a `SUPER_ADMIN`, `ADMIN` y `BOARD_OF_DIRECTORS`. El correo de contacto visible en pie y página de soporte se resuelve con `getPublicSupportEmail()`: prioriza el campo `support_email` de `platform_recurrente_settings`, luego `NEXT_PUBLIC_SUPPORT_EMAIL` y finalmente `EMAIL_FROM`.

7. Aspectos legales y contenido normativo

El texto legal del software se mantiene en `src/content/software-terms-document.ts` con versión referenciada desde `src/lib/software-terms.ts`. Las rutas `/legal/terms`, `/legal/legal-notice` y `/legal/software-terms` comparten el componente `SoftwareTermsBody` para garantizar consistencia. En registro se audita aceptación mediante `software_terms_accepted_at` y `software_terms_version` en la tabla `users`. Los pagos de cuota de plataforma pueden registrar `terms_accepted_at` y `terms_version` en `platform_fee_payments`.

8. Modelo de datos (Prisma)

Principales modelos: `User`, `Complex`, `Unit`, `Resident`, `Amenity`, `Reservation`, `Service`, `UnitService`, `Invoice`, `InvoiceItem`, `VisitorLog`, `Announcement`, `Event`, `EventRSVP`, `Incident`, `IncidentComment`, `Document`, `Poll`, `PollOption`, `Vote`, `PlatformFeePayment`, `PlatformRecurrenteSettings`. Los enums incluyen roles de usuario, tipos de complejo, estados de factura, estados de pago de plataforma, etc. La fuente canónica es `prisma/schema.prisma`. El motor configurado es MySQL en Railway (uso gratuito limitado a 30 días).

8.1 Módulos de interfaz (App Router)

Áreas públicas: inicio, login, registro, recuperación y restablecimiento de contraseña, páginas legales (`/legal/terms`, `/legal/legal-notice`, `/legal/software-terms`), soporte acotado por rol (`/support`). Área autenticada bajo `/dashboard`: panel principal, perfil y ajustes, complejos (listado, alta, ficha y edición), unidades y residentes (con fichas detalladas), amenidades y reservas, servicios, facturas y cobros internos de condominio, incidentes con hilos de comentarios, comunicaciones, anuncios (listado y editor), eventos y RSVP, encuestas y votación, documentos, control de acceso / visitantes, huéspedes Airbnb cuando aplique, informes, suscripción de plataforma (`platform-subscription`, `platform-payments`, `platform-recurrente`), flujo `success/cancel` tras pagos.

8.2 Superficie HTTP API (Route Handlers)

Endpoints agrupados por dominio bajo `src/app/api/`: autenticación `NextAuth` en `auth/[...nextauth]`; `auth/me` y perfil de usuario; CRUD y operaciones sobre `complexes` (`settings`, `units`, `announcements`, `events`,

incidents); units batch y unit-services; residents con reset de contraseña y perfil Airbnb; staff; amenities y reservations; services; announcements y polls con votación; events con RSVP; incidents y comments; invoices con generate y payment-intent; documents upload/download; visitors; payments checkout y webhook; bloque platform-fee (checkout, status, access-for-session, my-payments, sync-after-card-return, release-pending-card, admin payments confirm/reject/reconcile/manual-extend); platform/recurrente-config (GET/PUT para súper admin); recurrente fee-rates; notificaciones subscribe/test/unsubscribe; cron/billing; dashboard/stats; uploads relacionados según rutas activas.

9. Integraciones externas

- Recurrente: creación de checkout para suscripción de plataforma, webhooks y sincronización de platform_paid_hasta.
- Railway: hosting del servidor MySQL consumido por Prisma; la capa gratuita del proveedor está limitada a 30 días.
- Brevo (SMTP): envío transaccional mediante Nodemailer (recuperación de contraseña, notificaciones); versión gratuita del servicio.
- AWS S3: URLs firmadas / almacenamiento de documentos u objetos según implementación.
- reCAPTCHA: formularios públicos donde esté cableado.

10. Variables de entorno (referencia)

No se deben versionar valores secretos. Típicamente: DATABASE_URL (MySQL en Railway; período gratuito de 30 días), NEXTAUTH_SECRET, NEXTAUTH_URL, credenciales SMTP, claves PLATFORM_* / RECURRENT_* como respaldo de Recurrente, variables S3, y NEXT_PUBLIC_* solo para datos no sensibles expuestos al navegador.

11. Operaciones y respaldo

El script npm run db:backup ejecuta scripts/backup-db.cjs y escribe volcados SQL en prisma/backups/. Los backups pueden contener datos personales y no deben comprometerse ni confirmarse en control de versiones.

Carpeta compartida en Google Drive (solo equipo autorizado) con material de entorno de pruebas: archivo de variables .env (datos sensibles: credenciales, claves API, URLs con secretos; no difundir, no versionar en git) y copia SQL de base de datos con datos de demostración. Enlace:

<https://drive.google.com/drive/folders/1-95W6JsrA67q5hN7rGtXCzPF73pxy4mE?usp=sharing>

12. Inicialización del proyecto (primera vez)

Prerrequisitos: Node.js recomendado 20.x LTS (o versión compatible con Next.js del package.json), npm, y un servidor MySQL accesible (URL típica en DATABASE_URL).

- Clonar el repositorio y situarse en la raíz del proyecto.
- Instalar dependencias: npm install. El script postinstall ejecuta prisma generate.
- Crear archivo .env en la raíz (no versionar). Como mínimo local: DATABASE_URL con cadena mysql://... hacia una base vacía o dedicada;
 - variable de secreto para sesiones Auth.js/NextAuth (p. ej. AUTH_SECRET generada aleatoriamente; en algunos entornos también se usa NEXTAUTH_SECRET);
 - NEXTAUTH_URL con la URL pública del sitio (en local suele ser http://localhost:3000). Opcional

pero habitual: SMTP (SMTP_HOST, puerto, usuario, contraseña), EMAIL_FROM;

- NEXT_PUBLIC_APP_URL para redirects y enlaces externos; más variables según docs internas (.env debe seguir ignorado por git).
- Crear tablas: npx prisma migrate deploy aplicando prisma/migrations/ sobre la base indicada por DATABASE_URL. En desarrollo también puede usarse prisma migrate dev si el equipo crea nuevas migraciones locales.
- Datos de demostración (opcional): npx prisma db seed — crea usuarios de ejemplo definidos en prisma/seed.ts (revisar allí emails y contraseña por defecto; cambiar antes de exponer internet).
- Arranque en desarrollo: npm run dev (predev ejecuta prisma generate). Navegar a http://localhost:3000 y prefijo de idioma /es/ o /en/ según rutas definidas.
- Producción: npm run build y npm run start tras variables de entorno correctas. En entornos reales puede usarse npm run set-super-admin-password para definir una contraseña segura para el súper administrador después del seed.

13. Construcción y despliegue

- postinstall / predev ejecutan prisma generate para alinear el cliente con schema.prisma.
- npm run build produce artefacto de producción Next.js.
- Tras cambios de columnas Prisma: migraciones Prisma y prisma generate antes de desplegar.

14. Riesgos y líneas de mejora

- Planificar migración o suscripción en Railway antes de vencer los 30 días gratuitos del MySQL hospedado.
- Tener presentes los límites del plan gratuito de Brevo (SMTP) antes de picos de registros o campañas masivas.
- Auditar periódicamente roles y permisos en nuevas rutas API.
- Mantener backups cifrados fuera del entorno de desarrollo.
- Documentar Runbooks de webhook Recurrente y rotación de claves.

15. Anexo A: fuentes de producto ADESSO-365

Este anexo resume dónde reside la marca, textos y activos. El detalle tabulado vive en el repositorio en docs/adesso/fuentes-producto-adesso.md (índice en docs/adesso/README.md).

- public/logoadesso.svg y public/manifest.json: icono PWA, favicon y nombre corto ADESSO-365.
- src/app/[locale]/layout.tsx: título, metadatos PWA, tema, Inter (Google Fonts) y Malik (CDN).
- src/app/[locale]/(auth)/layout.tsx: hero ADESSO-365 y pie PropTech Solutions.
- src/messages/es.json y en.json: cadenas UX y legales con la marca.
- src/content/software-terms-document.ts, src/lib/software-terms.ts y SoftwareTermsBody.tsx: texto legal único.
- src/lib/email/send-password-reset.ts y src/lib/actions/auth-actions.ts: plantillas y nombre de marca en correo.
- src/lib/utils/pdf-generator.ts: pies y cabeceras en PDF exportados desde la aplicación.

16. Anexo B: base de datos

La persistencia oficial está descrita en `prisma/schema.prisma` (MySQL; conexión `DATABASE_URL`). Las migraciones versionadas están en `prisma/migrations/`. El detalle tabular Modelo !” tabla MySQL, listado de enums y política de backups figura en `docs/adesso/base-de-datos.md`.

Tablas físicas (@map): `users`, `complexes`, `platform_fee_payments`, `platform_recurrente_settings`, `units`, `residents`, `amenities`, `reservations`, `services`, `unit_services`, `invoices`, `invoice_items`, `visitor_logs`, `announcements`, `events`, `event_rsvps`, `incidents`, `incident_comments`, `documents`, `polls`, `poll_options`, `votes`. El respaldo operativo es `npm run db:backup !' prisma/backups/`; no debe versionarse SQL con datos sensibles ni el `.env`. La carpeta compartida de Google Drive descrita en §11 incluye plantilla `.env` confidencial y volcado SQL de prueba; antes de restaurar el SQL revisar compatibilidad con `prisma/schema.prisma`. Enlace repetido aquí por comodidad: <https://drive.google.com/drive/folders/1-95W6JsrA67q5hN7rGtxCzPF73pxy4mE?usp=sharing>.